

Agent changes — this session

Every change to the agent since the pipeline baseline (26fa7df, public 0.859), by author, with the before/after and the measured effect. The evaluator, catalog, public labels and API contract were **not** touched.

Score progression

checkpoint	who	public	adversarial (96)	note
pipeline baseline	team	0.859	0.684	dual-track + span rerank + FixedPolicy
+ windowed recommendations	KW	0.891	0.764	superseded by the elimination scan
+ tuned BM25 field weights	corainexia	0.898	0.725	merged onto the scan
+ elimination scan (turn 3)	KW	0.898	0.725	(this is what main carried)
+ facet-agreement rerank signal	corainexia	0.903	0.788	PR #3
+ retrieval-recall fixes	KW	0.903	0.788	this branch; dev 0.909, holdout 0.895

Net: **public 0.859 -> 0.903, adversarial 0.684 -> 0.788**. 39/39 tests pass. The four core-agent changes are detailed below; supporting tooling and docs follow.

Change 1 — Recommendation strategy: elimination scan (Kwong Weng)

Files: starter/agent.py, tools/sweep.py

Problem

After ~turn 3 the simulated customer has disclosed all ~4 constraints and the ranking stops changing. The old agent showed the **same top 10 every turn 3-10**, so any target not in that top 10 was a guaranteed miss — 12 of 200 public sessions, and far more on generic-constraint targets.

What changed

Two iterations, both gated by AgentConfig flags so the old behaviour is one switch away:

1. **Windowed scan** (b71db6d): freeze the reranked pool on the first list

shown, then walk it in windows — turn 3 ranks 1-10, turn 4 ranks 11-20, turn 5 ranks 21-30, ... Re-snapshot on a real new constraint or an intent override. AgentConfig.scan_windows.

2. **Elimination scan** (96d5385, ships): a product shown and not hit on is a

confirmed non-target (the session would have ended). So each turn: drop everything already shown, return the top 10 of the re-ranked **survivors**. No frozen pool, no cursor — rerank() runs every turn anyway, so new constraints and overrides are reflected automatically, and a target that drifts down the ranking is still a survivor and surfaces a turn later (no gap).

```
# starter/agent.py _shortlist(), simplified
shown = self._shown.setdefault(sid, set())
if (state.override_turn or 0) != self._shown_override.get(sid, 0):
    shown.clear() # a pre-override list confirms nothing
    self._shown_override[sid] = state.override_turn or 0
picks = [asin for asin, _ in candidates if asin not in shown][:limit]
shown.update(picks)
```

New AgentConfig fields: elimination_scan = True, hold_until_stalled = False. first_recommend_turn stays the start-turn knob; a dev+holdout sweep (tools/sweep.py: elim1/2/3, elim_hold) picked **turn 3** — earlier starts freeze a poor MRR on under-informed early hits.

One correctness fix

An intent-override session's pre-override list often already contains the target, but the evaluator ignores hits until the override lands. Without clearing the shown-set on override detection the scan sails past it — this collapsed override to 0.11 in one iteration; the clear restores it to 1.00.

Effect

Public 0.891 (windowing) then elimination $\approx$ same on public but **fixes the boundary-MRR regression** (0.756 -> 0.883) and is simpler. Public hit@10 0.940 -> 0.990; only 2 misses left. On the adversarial set the two variants trade (windowing 0.764, elimination 0.725) — elimination re-ranks every turn, so on all-generic constraints the "no preference" junk spans jostle a borderline target away. Change 4 fixes that.

Change 2 — Tuned BM25 field weights (corainexia, PR #2)

File: src/index.py commit 201c00d

```
DEFAULT_WEIGHTS (parent_asin, title, categories, features, details, store, description)
before (0.0, 6.0, 4.0, 2.5, 2.5, 1.5, 1.0)
after (0.0, 8.0, 5.0, 6.0, 6.0, 0.5, 0.25)
```

Heavier title / categories / features / details; store and long marketing description almost silenced. The original tuple was inherited from the starter and had never been tuned.

Effect

Public 0.8928 -> **0.8982** (+0.005), almost entirely MRR (0.804 -> 0.821) — hit@10 was already saturated, so this ranks the found target higher. Every scenario's MRR improved; holdout rose more than dev, so it generalises.

Change 3 — Facet-agreement rerank signal (corainexia, PR #3)

File: src/rerank.py commit 2d7f92b

Adds a fourth term to the rerank score. `_facet_agreement()` runs `src/facets.py:extract()` over the customer's accumulated text and over each candidate, and counts matching facet values (material, colour, size, style, use_case):

```
total = ( span_weight * coverage
+ retrieval_weight * (bm25 / max_bm25)
+ popularity_weight * popularity
+ facet_weight * facet_score ) # facet_weight = 0.3 <-- new
```

This is docs/ideas.md Idea 2 (partial) — the reranker had been ignoring FacetStore, which is built for every product but read by nothing.

Effect

Public 0.8995 -> **0.9031**; recovers the MRR that Change 4's query filter costs on its own and adds more (public MRR 0.814 -> 0.825).

Change 4 — Retrieval recall: hold declines out of the query (Kwong Weng)

Files: src/state.py, src/rerank.py commit 960eed0

Problem

`retrieve()` queries `state.full_text()` = every utterance joined. From turn 4 the simulator's "*I don't have an additional preference for feature / use_case / style / material / colour / size*" leaks feature, style, material, colour, size, category into the BM25 OR-query and the span matcher — terms that match huge swathes of the catalog and dilute the target. Recall **degrades within a session**: adversarial "target in the 300-pool" fell from 96% (turn 3) to 81% (turn 10).

What changed

- `Utterance.declined` flag, set in `observe()` when `NO_PREFERENCE_CUES`

`matches.full_text()`, `focused_text()` and `query_spans()` skip declined utterances; a decline also no longer counts as a "productive" turn.

- `RerankConfig.depth` **200 -> 300** — rescore the whole retrieval

pool, not a prefix. ~12% of cluster-target sessions had the target in the pool but past rank 200, where the span signal never applied. (No-op on public; helps only the adversarial set.)

Effect — recall (the point of the change)

target in 300-pool at end of session	before	after
public	98%	100%
adversarial	81%	95%
adversarial: degenerate_card / homogeneous_cluster	56% / 81%	88% / 100%
end-of-session pool-rank p90 (public / adversarial)	87 / 279	16 / 200

The within-session degradation is gone. Score: adversarial 0.725 -> **0.788** (hit@10 0.802 -> 0.885); public flat on its own, net **+0.005** once combined with Change 3.

Supporting work (Kwong Weng)

file	what
<code>tools/hard_cases.py</code>	Adversarial session generator + per-bucket scorer. Scans the frozen catalog, buckets every product by an adversarial property, samples 16 each. <code>--run</code> scores the agent grouped by bucket.
<code>data/hard_set.jsonl</code>	96 generated sessions (6 buckets: <code>homogeneous_cluster</code> , <code>budget_only_signal</code> , <code>boilerplate_soft</code> , <code>degenerate_card</code> , <code>generic_override</code> , <code>cross_category_collision</code>). Public-set schema; scored by the unmodified evaluator.
<code>tools/sweep.py</code>	<code>build_configs()</code> — added <code>plain</code> , <code>elim1/2/3</code> , <code>elim_hold1/2</code> for the start-turn sweep.
<code>docs/ideas.md</code> / <code>ideas.pdf</code>	Reranking & recommendation-strategy ideas, each with the measured result: elimination scan (1a/1b), decline filter (1c), facet / category / MMR / learned-weights (2-6).
<code>docs/hard_cases.md</code> / <code>.pdf</code>	Failure analysis of the adversarial set and the prioritised fix plan.
<code>agent_summary.pdf</code>	Rewritten (c7757af) for the current elimination-scan workflow: the loop, one turn stage-by-stage, recommendation timing, and a round-by-round table per scenario.
<code>examples.pdf</code>	Two annotated agent<->simulator transcripts (one hit, one miss).

Also merged this session (teammates, non-agent)

commit	author	what
a13d8b5	Elinengu	<code>Customer_Simulator_Explainer.pdf</code>
80bd4f8	Elinengu	<code>tools/observe.py</code> , <code>tests/test_observe.py</code> — session observer / monitor
0f085ec	Elinengu	<code>docs/intent_override_retrieval.{md,pdf}</code>
01ffdcdb	Elinengu	<code>docs/reranking_explained.{md,pdf}</code>

Current agent settings (after all changes)

stage	knob	value	file
S1 index	FTS5 field weights (title, cat, features, details, store, desc)	8, 5, 6, 6, 0.5, 0.25	src/index.py
S5 retrieval	pool size / RRF k / focused-route weight	300 / 60 / 0.8	src/retrieval.py
S6 rerank	span / retrieval / popularity / facet weight	1.0 / 1.0 / 0.02 / 0.3	src/rerank.py
S6 rerank	length bonus / depth	0.12 / 300	src/rerank.py
S3 state	pre-override utterance weight	0.35 (largely inert)	src/state.py
S3 state	declined utterances held out of every retrieval view	—	src/state.py
S4 policy	FixedPolicy: other, then feature-ladder	—	src/policy.py
S7 timing	first_recommend_turn / confidence margin / earliest	3 / 0.20 / 2	starter/agent.py
S7 timing	elimination_scan / hold_until_stalled	on / off	starter/agent.py

Not touched (organizer-owned)

evaluator/local_evaluator.py, data/catalog.jsonl, data/public_set.jsonl, docs/agent_api_contract.json, docs/evaluation_config.json.